ESCUELA DE INGENIERÍA

Programa Ingeniería de Sistemas

Modalidad – Presencial

🏁 ***Proyecto ABP: Arquitectura distribuida, ETL y procesamiento con Mojo***

Taller teórico-práctico
Modalidad: Individual o en equipos
Entregable final: archivo.bat o archivo.sh

1. 🎯 PROPOSITO DEL PROYECTO

En este proyecto el estudiante asumirá el rol de **ingeniero de datos**, encargado de diseñar e implementar una solución para integrar información proveniente de diferentes fuentes de datos distribuidas.

La solución deberá implementar una arquitectura basada en **contenedores Docker**, bases de datos MongoDB distribuidas por dominios de información, una base de datos relacional y procesos de extracción, transformación y carga (**ETL**).

Como elemento diferenciador, las funciones principales de procesamiento y transformación deberán implementarse utilizando **Mojo, el lenguaje de programación desarrollado por Modular**.

Mojo está diseñado para programación de alto rendimiento y permite interoperabilidad con el ecosistema Python, lo que posibilita integrarlo en soluciones de procesamiento de datos.

El proyecto busca que el estudiante no solamente ejecute instrucciones, sino que pueda **comprender, construir, justificar y demostrar una arquitectura de datos funcional**.

2. 📚 SITUACIÓN PROBLEMA

Una organización dispone de información correspondiente a:

- Personas.
- Artículos.
- Ventas.

Los datos se encuentran inicialmente distribuidos en archivos JSON independientes.

La organización requiere implementar una solución que permita:

1. Distribuir la información en diferentes nodos.
2. Implementar cada nodo mediante Docker.
3. Almacenar la información en MongoDB.
4. Consultar la información distribuida.
5. Identificar y corregir inconsistencias en los datos.
6. Implementar procesos ETL.
7. Transformar los datos utilizando Mojo.
8. Integrar la información en una base de datos relacional.
9. Mantener la integridad referencial.
10. Realizar consultas analíticas.
11. Generar visualizaciones que permitan interpretar los resultados.

El reto

¿Cómo diseñar e implementar una arquitectura distribuida de datos que permita integrar información heterogénea, aplicar procesos ETL utilizando Mojo y garantizar que los datos finalmente almacenados en una base de datos relacional sean consistentes, trazables y analizables?

3. MATERIAL SUMINISTRADO

Se entregara únicamente:

3.1 Cuaderno guía: proyectoBDM.ipynb

- Este archivo constituye la **guía oficial de desarrollo del proyecto**.
- El estudiante deberá completar las secciones, instrucciones y espacios establecidos.

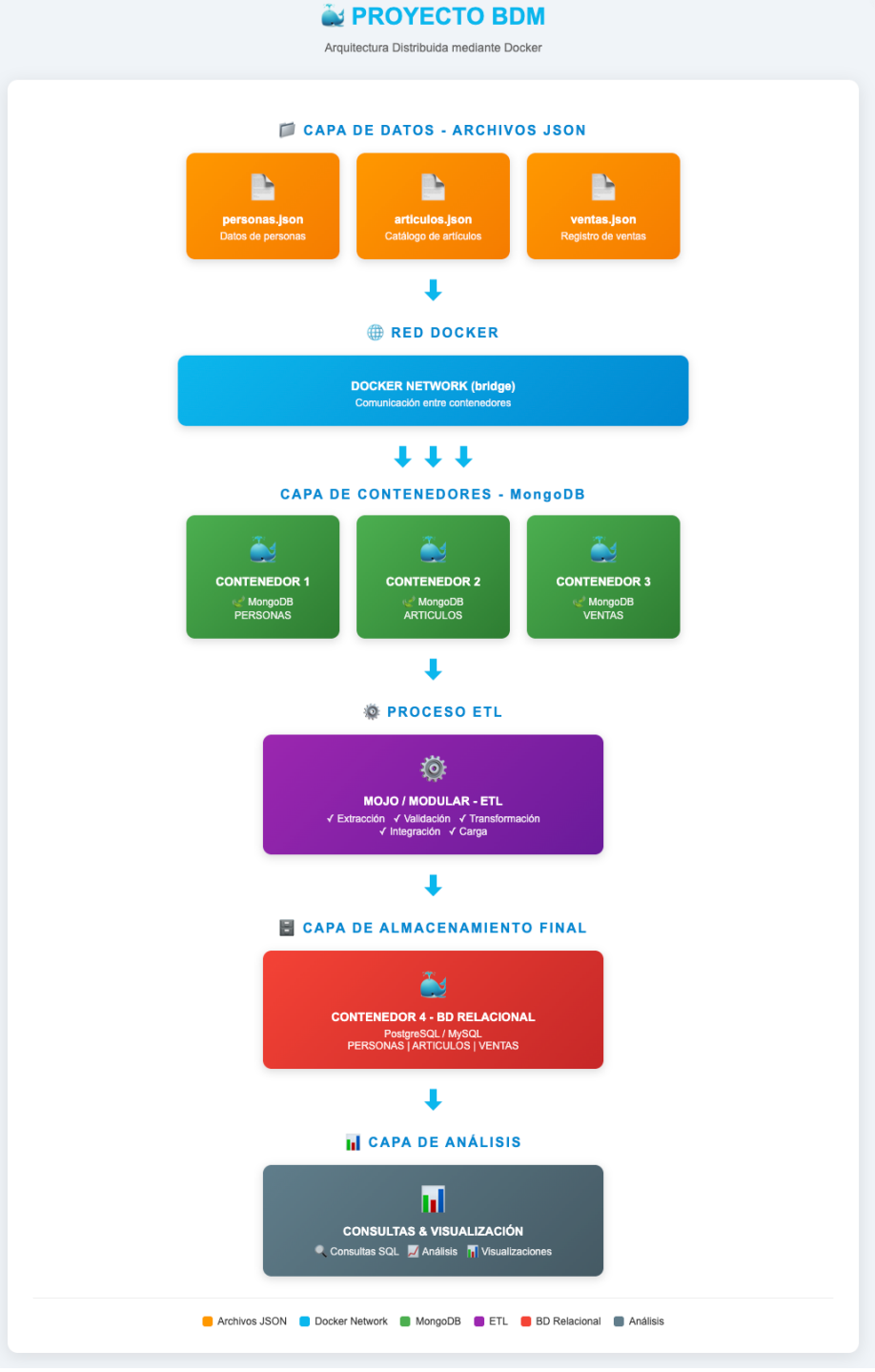
3.2 Se entregarán los siguientes Datos:

- personas.json
- articulos.json
- ventas.json

Los estudiantes **no deberán modificar los archivos originales**. Los datos contienen intencionalmente diferentes situaciones de calidad de datos que deberán ser identificadas y tratadas durante el proceso ETL.

3.3 Se suministrará el archivo formato.sql representa la estructura relacional.

3.4 Diagrama inicial Se entregará un diagrama de referencia de la arquitectura esperada. El estudiante deberá **completarlo, interpretarlo y justificarlo** dentro de su IPYNB.



4. 📚 ARQUITECTURA ESPERADA

La solución deberá implementar una arquitectura distribuida mediante Docker.

Nota conceptual

La arquitectura corresponde a una **arquitectura distribuida y heterogénea de datos**. No debe confundirse con un clúster MongoDB con replicación automática.

Cada dominio de información se encuentra aislado en su propio nodo/contendor MongoDB y posteriormente la información es integrada mediante ETL.

5. ✏️REQUISITO FUNDAMENTAL: MOJO

El proyecto deberá utilizar **Mojo de Modular**

NO se acepta

- Mojolicious.
- Perl como sustituto de Mojo.
- Código generado completamente en Python para reemplazar las funciones solicitadas en Mojo.
- Simular que se utilizó Mojo sin evidencia de ejecución.

SÍ se requiere

Los procesos fundamentales de transformación deberán implementarse en archivos .mojo.

Mojo permite crear módulos y paquetes reutilizables, por lo que se espera que el estudiante organice su solución de manera modular.

6. 🎨 FUNCIONES MÍNIMAS QUE DEBERÁN DESARROLLARSE EN MOJO

Como mínimo, el proyecto deberá implementar funciones equivalentes a:

- **Limpieza numérica**

limpiar_valor_numerico()

- Responsable de identificar y limpiar valores que deberían ser numéricos.

- **Validación de personas**

validar_persona()

Deberá validar, entre otros:

- numeroDocumento
- telefono

- **Transformación de artículos**

limpiar_articulo()
Deberá validar:

- idArticulo
- precioArticulo
- cantidadArticulo

- **Transformación de ventas**

limpiar_venta()
Deberá validar:

- idComprador
- idArticulo
- cantidadProductos
- precioTotal

- **Integridad referencial**

validar_integridad_referencial()
Deberá comprobar que:

- Los compradores de las ventas existan en PERSONAS.
- Los artículos de las ventas existan en ARTICULOS.
- Los identificadores sean compatibles con el modelo relacional.

Proceso ETL
ejecutar_etl()
Deberá coordinar:

EXTRACT
↓
VALIDATE
↓
TRANSFORM
↓
LOAD

7. CONDICIONES ESPECIALES SOBRE PYTHON

Python podrá utilizarse dentro del IPYNB para:

- Ejecutar comandos Docker.
- Realizar solicitudes HTTP.
- Mostrar resultados.
- Generar DataFrames.
- Construir gráficos.
- Interactuar con el entorno Jupyter.

Sin embargo:

Las funciones centrales de limpieza, validación, transformación e integración solicitadas en el proyecto deberán estar implementadas en Mojo.

Mojo dispone de interoperabilidad con Python, por lo que esta integración puede ser utilizada cuando sea técnicamente necesaria.

El estudiante deberá explicar claramente qué parte realiza Python y qué parte realiza Mojo.

8 DESARROLLO DEL PROYECTO

8.2 Fase 1. Identificación del problema

El estudiante deberá explicar:

- Problema planteado.
- Fuentes de datos.
- Características de los datos.
- Riesgos de calidad.
- Necesidad de distribución.
- Necesidad de integración.
- Justificación de la arquitectura.

8.3 Fase 2. Arquitectura Docker

El estudiante deberá:

1. Crear la red Docker.
2. Crear los contenedores.
3. Configurar los servicios.

4. Establecer comunicación entre ellos.
5. Definir puertos cuando sea necesario.
6. Verificar que todos los contenedores estén activos.

Deberá existir evidencia de: docker ps y de la comunicación entre los servicios.

8.4 Fase 3. Distribución MongoDB

Cada contenedor MongoDB tendrá una responsabilidad:

Contenedor	Base/colección
MongoDB 1	PERSONAS
MongoDB 2	ARTICULOS
MongoDB 3	VENTAS

Cada archivo JSON deberá ser cargado en su correspondiente distribución.

El estudiante deberá demostrar:

- Carga exitosa.
- Número de documentos.
- Estructura de documentos.
- Ejemplos de registros.
- Identificación de inconsistencias.

8.5 Fase 4. Análisis de calidad de datos

Antes de cargar los datos en la base relacional, el estudiante deberá identificar problemas.

Por ejemplo: @10@93@!65, q61@A!213A, 2x

El estudiante deberá explicar:

1. Qué problema presenta el dato.
2. Qué regla de validación aplica.
3. Qué transformación realiza.
4. Qué ocurre si el dato no puede recuperarse.
5. Cómo evita que el error llegue a la base relacional.

No será suficiente con eliminar registros sin explicar la decisión.

8.6 Fase 5. ETL utilizando Mojo

El estudiante deberá implementar:

Extract Obtener información desde los nodos MongoDB.

Transform Aplicar:

- Limpieza.
- Conversión de tipos.
- Validación.
- Normalización.
- Eliminación de campos técnicos de MongoDB cuando corresponda.
- Validación de claves.

Load

Insertar los datos válidos en:

- PERSONAS
- ARTICULOS
- VENTAS

de la base de datos relacional.

8.7 Fase 6. Integridad referencial

La solución deberá garantizar:

PERSONAS.numeroDocumento
▲
VENTAS.idComprador

ARTICULOS.idArticulo
▲
VENTAS.idArticulo

El estudiante deberá demostrar mediante consultas que no existen ventas huérfanas.

8.8 Fase 7. Consultas y análisis

El IPYNB deberá desarrollar, como mínimo, los siguientes análisis:

- **Cinco artículos más vendidos**
  Mostrar y graficar los cinco artículos con mayor cantidad de unidades vendidas.
- **Cinco compradores con mayor número de compras**
  Mostrar y graficar los cinco compradores que hayan realizado mayor cantidad de compras.
- **Distribución de precios**
  Analizar y visualizar la distribución de precios de los cinco primeros artículos solicitados por el cuaderno.
- **Cinco artículos menos vendidos**
  Identificar y graficar los cinco artículos con menor cantidad de unidades vendidas.

9 EVIDENCIA OBLIGATORIA

Cada resultado importante deberá estar acompañado por evidencia.

Por ejemplo:

Código
↓
Ejecución
↓
Resultado
↓
Interpretación
No se aceptará únicamente: "El resultado es correcto." El estudiante deberá explicar **qué significa el resultado**.

10 ESTRUCTURA OBLIGATORIA DEL IPYNB

El estudiante deberá entregar un único archivo: ApellidoNombre_ProyectoBDM.ipynb

El notebook deberá contener, como mínimo:

1. Identificación del grupo

- NRC.
- Nombres.
- ID.
- Fecha.

2. Introducción
Descripción del problema.

3. Arquitectura
Diagrama y explicación.

4. Docker
Creación, configuración y evidencia de contenedores.

5. MongoDB
Carga, consultas y conteo de documentos.

6. Calidad de datos
Identificación de inconsistencias.

7. Mojo
Código y explicación de las funciones implementadas.

8. ETL
Extract → Transform → Load.

9. Base de datos relacional
Carga de:

- PERSONAS.
- ARTICULOS.
- VENTAS.

10. Integridad referencial
Validaciones y resultados.

11. Consultas SQL
Consultas requeridas.

12. Visualizaciones
Gráficos solicitados.

13. Análisis
Interpretación de resultados.

14. Conclusiones
Reflexión técnica del equipo.

15. Referencias
Documentación oficial utilizada.

11 🧩 REQUISITOS DE EJECUCIÓN

El IPYNB deberá poder ejecutarse **de principio a fin**.

El estudiante deberá evitar rutas absolutas como: /Users/nombre_usuario/... o: C:\Users\...

Deberán utilizarse rutas relativas o dinámicas que permitan reproducir el proyecto en otro equipo.

12 REQUISITOS DE EJECUCIÓN

Una solución será considerada reproducible cuando:

1. El proyecto pueda iniciarse desde el entorno indicado.
2. Los contenedores puedan levantarse.
3. Los datos puedan cargarse.
4. Las funciones Mojo puedan ejecutarse.
5. El ETL pueda ejecutarse.
6. La base relacional pueda poblarse.
7. Las consultas puedan ejecutarse.
8. Los gráficos puedan generarse.

13 EVIDENCIAS DE MOJO

El estudiante deberá incluir en el IPYNB:

- Código fuente Mojo.
- Nombre de los módulos.
- Funciones implementadas.
- Ejecución.
- Resultados.
- Explicación de las funciones.
- Versión de Mojo utilizada.

También deberá demostrar la ejecución mediante comandos de la CLI de Mojo cuando corresponda. La CLI oficial permite ejecutar, compilar, depurar y empaquetar programas Mojo.

14 USO DE INTELIGENCIA ARTIFICIAL

La inteligencia artificial puede utilizarse como **herramienta de apoyo al aprendizaje**, pero no como sustituto del trabajo intelectual del estudiante.

Se permite utilizar IA para:

- Comprender conceptos.

- Consultar sintaxis.
- Detectar errores.
- Proponer alternativas.
- Comparar soluciones.

Sin embargo, el estudiante deberá poder explicar y defender oralmente el funcionamiento de su solución.

Regla de autoría

Los aportes generados exclusivamente por inteligencia artificial, sin comprensión, adaptación, validación y explicación por parte del estudiante, podrán recibir una calificación de 0.0 en el componente correspondiente.

La utilización de IA deberá declararse al final del IPYNB.

Ejemplo:

- Herramienta de IA utilizada:
- Propósito:
- Qué parte fue utilizada:
- Qué modificaciones realizó el equipo:
- Cómo fue validada:

🧩 15 DEFENSA DEL PROYECTO

El docente podrá seleccionar aleatoriamente cualquier función, consulta o componente del proyecto y solicitar a un estudiante:

- Explicarlo.
- Ejecutarlo.
- Modificarlo.
- Corregirlo.
- Justificar una decisión.

Por ejemplo: "Cambie la regla de validación de numeroDocumento." o: "Explique qué sucede si una venta referencia un artículo inexistente." o: "Modifique la consulta para obtener los 10 artículos más vendidos."

Por esta razón, **el IPYNB debe representar realmente el trabajo desarrollado por el equipo**.

16 ENTREGABLE ÚNICO

Estos elementos serán construidos/utilizados durante el desarrollo, pero la evidencia académica se concentrará en un único documento. **El estudiante deberá entregar exclusivamente:**

ApellidoNombre_ProyectoBDM.ipynb

El IPYNB deberá estar:

- Completo.
- Ejecutado.
- Ordenado.
- Documentado.
- Con resultados visibles.
- Con gráficos.
- Con código.
- Con explicaciones.
- Con referencias.

17 RÚBRICAS DE EVALUACIÓN

Criterio	Peso
Arquitectura distribuida y Docker	15%
Implementación de MongoDB	10%
Calidad y transformación de datos	15%
Implementación de funciones en Mojo	20%
Proceso ETL	15%
Integridad de la base relacional	10%
Consultas y visualizaciones	10%

Criterio	Peso
Análisis, documentación y referencias	5%
TOTAL	**100%**

Escala

Nivel	Descripción
5.0 – Excepcional	Solución completa, funcional, reproducible, técnicamente sólida y correctamente argumentada.
4.0 – Alto	Cumple ampliamente los requisitos con errores menores.
3.0 – Esperado	Cumple los requisitos fundamentales, aunque presenta oportunidades de mejora.
2.0 – Bajo	Cumplimiento parcial, errores importantes o dependencia de soluciones no justificadas.
1.0 – Deficiente	No demuestra dominio del proyecto o presenta evidencia insuficiente.

🧩 18 CRITERIOS DE CALIDAD SUPERIOR

Un proyecto podrá destacarse cuando, además de cumplir los requisitos mínimos:

- Utilice una arquitectura modular en Mojo.
- Maneje adecuadamente errores.
- Evite duplicación de código.
- Documente las funciones.
- Realice validaciones adicionales.
- Analice la calidad de los datos.
- Justifique decisiones técnicas.
- Presente visualizaciones relevantes.
- Demuestre reproducibilidad.

- Explique las decisiones de arquitectura.
- Proponga mejoras futuras.

🧩 19 PREGUNTA DE CIERRE

Al finalizar el proyecto, el equipo deberá responder:

¿Qué ventajas y limitaciones encontraron al integrar MongoDB, una base de datos relacional, Docker y Mojo dentro de una arquitectura distribuida para procesamiento de datos?

La respuesta deberá estar sustentada en la experiencia real obtenida durante el desarrollo.

Resultado esperado

- Al finalizar el proyecto, el estudiante deberá ser capaz de demostrar que puede: **Diseñar → Distribuir → Almacenar → Validar → Transformar → Integrar → Consultar → Analizar**
- información masiva utilizando diferentes tecnologías dentro de una misma arquitectura.
- **El objetivo no es solamente que el programa funcione, el objetivo es que el estudiante pueda explicar por qué funciona, cómo funciona y qué decisiones de ingeniería tomó para construirlo.**

🎯 **Mensaje:**

"Una buena arquitectura de datos no es la más compleja, sino la que resuelve el problema del negocio de forma escalable, mantenible y costo-eficiente. El valor como futuro arquitecto no está en conocer todas las herramientas, sino en saber elegir la adecuada para cada contexto."